

PredictQ

Founding Engineer Technical Challenge

2-Hour Live Build • Screenshare • Real-Time Evaluation

Overview

This is a deliberately ambitious challenge for a 2-hour window. We don't expect you to finish everything; we expect you to make smart decisions about what to build first and why. A working pipeline with rough edges beats a polished UI with no backend. Show us how you think and prioritize under pressure.

The Task: Market Intelligence Dashboard

Build a full-stack application that ingests, stores, and surfaces Polymarket prediction market data. Work through the requirements in order; get each layer functional before moving to the next.

1. Ingest live market data from Polymarket's public API on a recurring basis (poll every 30–60 seconds).
2. Store and normalize the data in a database with a schema that supports historical tracking (e.g., price changes over time).
3. Expose a backend API your frontend consumes, at minimum: GET /markets, GET /markets/:id, and GET /markets/:id/history.
4. Build a frontend that lets users interact with the data: filter, sort, search, or drill into a market.
5. Include at least one AI-powered feature that derives genuine insight from the stored data. Examples: market summarizer, trend detector, smart ranking, volatility alert.

Polymarket Public API — No Auth Required

Markets endpoint: <https://gamma-api.polymarket.com/markets>

No API key needed for read access. Returns question, outcomes, prices, volume, end date.

Use any LLM API for your AI feature (OpenAI, Anthropic, Grok, etc.).

Technical Requirements

- Stack is your choice. Use whatever you're fastest with.
- System must run end-to-end within 2 hours: ingestion → storage → API → frontend.
- Commit frequently; we review your commit history as part of the evaluation.
- Caching and near real-time updates (SSE/WebSockets) are bonuses if time permits

Submission

At the end of 2 hours, send two links:

Dimension	What We're Looking For
GitHub Repo	Public repo with your full commit history. A README is optional but appreciated.
Live URL	Something running we can interact with. Vercel, Railway, Fly.io, Render, etc. A localhost screenshot is not sufficient.

What We're Evaluating

Dimension	What We're Looking For
Working system	Does the full pipeline run end-to-end? Ingestion → storage → API → frontend. A rough working system scores better than a polished non-functional one.
Prioritization	What did you build first? What did you cut? Good engineers make deliberate tradeoffs under pressure.
System design	Is your schema sensible? Does your API layer make sense? We're looking for clean separation of concerns.
AI integration	Is the AI feature substantive and integrated with your data model? We're looking for product thinking, not just a prompt wrapper.
Code quality	Structure, error handling, edge cases. We read the code.
Process	Your commit history shows how you approached the problem. We value clear, iterative thinking.

Tips
Work in order: get ingestion + storage working first, then the API layer, then the frontend.
Deploy early, even if it's empty. If you don't have a preferred setup, Railway handles backend + database in one place and is the fastest path to a live URL.
Your AI feature choice reveals your product intuition. Pick something you'd actually want to use.
If something breaks, narrate what you're thinking. We're watching the screenshare.
If you run out of time, a working pipeline with a basic frontend beats a beautiful UI with no data behind it.

What happens next: We'll review your submission before the debrief call. The debrief is a conversation; we'll ask about the decisions you made, what you'd do with more time, and how you'd extend it.